
ELIANDATAAGENT: A PLAN-EXECUTE-REPLAN MULTI-AGENT FRAMEWORK WITH DAG-ENHANCED PARALLEL EXECUTION FOR DATA ANALYSIS TASKS

Zimo Li^{1*}

¹ School of Mathematics and Statistics, Nanjing University of Information Science and Technology
{lizimo}@nuist.edu.cn,

https://github.com/2Elia/elian_data_agent

ABSTRACT

Data analysis tasks in real-world scenarios often require complex multi-step reasoning, tool orchestration, and dynamic adaptation to intermediate results. Existing single-agent approaches struggle with task decomposition, parallel execution, and failure recovery, leading to inefficient workflows and limited scalability. We present EliaDataAgent, a multi-agent framework that integrates the Plan-Execute-Replan paradigm with Directed Acyclic Graph enhanced parallel execution for sophisticated data analysis tasks. EliaDataAgent employs three specialized agents: a Planner that decomposes tasks into structured execution plans with explicit dependencies, an Executor that performs step-wise reasoning using the ReAct pattern, and a Replanner that dynamically adjusts plans based on execution outcomes. The system further enhances execution efficiency through a DAG scheduler that supports three execution strategies: Sequential, Parallel, and Hybrid that enabling dependency-aware concurrent task processing. Experimental evaluation demonstrates that EliaDataAgent effectively handles diverse data analysis scenarios including CSV exploration, SQL querying, and multi-source data integration, achieving robust task completion with adaptive replanning capabilities. The framework provides a practical and scalable foundation for building intelligent data analysis agents that combine strategic planning with efficient parallel execution.

Keywords Multi-Agent Systems · Plan-Execute-Replan · DAG Execution · Data Analysis · ReAct Pattern

1 Introduction

Large language models (LLMs) have demonstrated remarkable capabilities in natural language understanding, code generation, and tool use [1, 2]. These advances have enabled a new generation of intelligent agents that can perform complex reasoning tasks, interact with external tools, and solve multi-step problems autonomously [4, 5, 6]. However, real-world data analysis tasks present unique challenges that exceed the capabilities of single-agent systems.

Data analysis workflows typically involve multiple interconnected operations: exploring file structures, reading diverse data formats (CSV, JSON, SQLite), formulating and executing SQL queries, performing statistical computations, and synthesizing final answers. These tasks exhibit several characteristics that complicate single-agent approaches:

- **Complex task decomposition:** High-level queries like “analyze the sales trends across multiple tables” require sophisticated planning to identify relevant data sources, determine appropriate analysis methods, and sequence operations correctly.
- **Dependency management:** Certain operations must precede others (e.g., schema inspection before query formulation), while independent operations could potentially execute concurrently for efficiency.
- **Dynamic adaptation:** Intermediate results may reveal unexpected patterns or data quality issues, requiring plan modifications without complete restarts.
- **Error recovery:** Failed operations (e.g., malformed queries, missing columns) need graceful handling with alternative approaches rather than task failure.

Existing approaches to LLM-based data analysis predominantly employ single-agent architectures with ReAct-style reasoning loops [4]. While effective for straightforward tasks, these systems face fundamental limitations: they cannot parallelize independent operations, struggle with complex dependency management, and lack systematic mechanisms for plan revision. Memory-enhanced agents [8, 9] improve context retention but do not address structural planning challenges. Specialized tool-learning methods [5, 37, 39] enhance individual tool use but lack orchestration frameworks for multi-tool coordination.

In this paper, we present **ElianDataAgent**, a multi-agent framework that addresses these challenges through two integrated design principles: the Plan-Execute-Replan (PER) paradigm and DAG-enhanced parallel execution. The framework draws inspiration from established agent architectures—specifically the Eino Agent Development Kit’s session-based agent design and Shannon’s DAG workflow patterns—while introducing novel adaptations for data analysis domains.

The ElianDataAgent architecture comprises three specialized agents operating within a shared Session context:

- **Planner Agent:** Decomposes user queries into structured execution plans with explicit step dependencies, supporting Sequential, DAG, and Hybrid execution modes.
- **Executor Agent:** Executes individual plan steps through ReAct-style reasoning (Think-Act-Observe loops) with access to a validated tool registry for data manipulation.
- **Replanner Agent:** Reviews execution outcomes and dynamically adjusts plans—adding, modifying, or removing steps based on intermediate results and error conditions.

Beyond agent specialization, ElianDataAgent introduces a DAG execution engine that enables dependency-aware parallel execution. Using Kahn’s algorithm for topological sorting and a three-mode execution strategy (Sequential, Parallel, Hybrid), the system maximizes throughput while respecting operation dependencies. A shared Workspace component manages cold-start schema discovery, file caching, and knowledge integration, providing all agents with consistent context about available data sources.

The main contributions of this paper are:

- We propose a multi-agent architecture that combines Plan-Execute-Replan orchestration with DAG-based parallel execution, enabling both strategic planning and efficient task processing for data analysis scenarios.
- We design a comprehensive type system for inter-agent communication, including Plan/PlanStep structures, Session state management, and History tracking mechanisms.
- We implement three execution strategies (Sequential, Parallel, Hybrid) with ThreadPoolExecutor-based concurrency control and incremental dependency checking.
- We develop a secure tool registry with eight validated data manipulation tools featuring path validation, SQL injection prevention, and execution timeouts.
- We provide a modular, extensible implementation compatible with OpenAI 0.x and 1.x APIs, suitable for both research experimentation and practical deployment.

2 Related Work

We organize related work into four research threads: multi-agent orchestration patterns, reasoning-acting frameworks, DAG-based task execution, and tool-augmented language models. ElianDataAgent synthesizes insights from all four directions while introducing novel adaptations for data analysis tasks.

2.1 Multi-Agent Orchestration Patterns

Multi-agent systems for LLM orchestration have emerged as a powerful paradigm for decomposing complex tasks into specialized subtasks. Early frameworks like AutoGen [?] demonstrated conversational agent collaboration through message passing, while BabyAGI [?] introduced task-driven execution loops with vector-based task storage. These systems highlighted the value of agent specialization but lacked structured dependency management.

The Plan-Execute-Replan (PER) pattern, formalized in systems like Plan-and-Execute agents [?], provides a principled approach to task decomposition and adaptive execution. In PER architectures, a planner generates structured execution plans, an executor processes individual steps, and a replanner dynamically adjusts plans based on outcomes. ElianDataAgent adopts this three-agent structure but extends it with DAG-based execution ordering and dependency-aware parallelization.

Session-based state management, inspired by frameworks like Eino ADK, enables consistent information sharing across agents through centralized KV stores. ElianDataAgent implements a Session component that maintains execution state, intermediate results (as topics), and plan history, providing a unified context for all agents.

2.2 Reasoning-Acting Frameworks

The ReAct (Reasoning + Acting) framework [4] introduced interleaved thinking and acting as a core paradigm for tool-using agents. In ReAct loops, agents alternate between generating thoughts (reasoning about current state), actions (tool invocations), and observations (tool outputs), enabling transparent reasoning traces and iterative refinement. This pattern has been widely adopted in agent benchmarks including WebShop [40], WebArena [42], and ALFWorld [41].

EliaDataAgent integrates ReAct-style reasoning within its Executor agent, enabling step-wise tool use with observable reasoning traces. However, unlike single-agent ReAct systems, EliaDataAgent confines ReAct loops to individual step execution, delegating plan-level decisions to the Planner and Replanner agents. This separation enables both fine-grained tool reasoning and macro-level plan management.

Chain-of-thought prompting [?] and its extensions (Tree of Thoughts [?], Graph of Thoughts [?]) improve reasoning quality but operate within single-agent contexts. EliaDataAgent’s multi-agent structure complements these approaches by distributing reasoning responsibilities across specialized agents.

2.3 DAG-Based Task Execution

Directed Acyclic Graphs (DAGs) provide a natural representation for tasks with dependencies, enabling both topological ordering for sequential execution and parallel execution of independent nodes. Work on DAG-based workflow systems—including Apache Airflow, Luigi, and Shannon’s DAG patterns [?]—has established principles for dependency management, cycle detection, and concurrent execution.

Kahn’s algorithm [?] remains a standard method for topological sorting with efficient cycle detection. EliaDataAgent employs Kahn’s algorithm in its DAGGraph component, computing execution layers that reveal which nodes can execute concurrently. The DAGExecutor implements three execution strategies:

- **Sequential:** Execute all nodes in topological order, suitable for tasks with tight dependencies.
- **Parallel:** Execute all nodes concurrently, applicable when no dependencies exist.
- **Hybrid:** Execute independent nodes concurrently while respecting dependencies, achieving optimal throughput.

Concurrency control through ThreadPoolExecutor with semaphore-based throttling prevents resource exhaustion while enabling genuine parallelism. Incremental dependency checking with configurable timeouts handles long-running dependencies gracefully.

2.4 Tool-Augmented Language Models

Tool use has become a core capability for LLM agents, enabling interaction with external APIs, databases, and computational resources. Toolformer [5] demonstrated self-supervised tool learning, while Gorilla [6] trained models specifically for API invocation. Tool-oriented benchmarks (API-Bank [38], ToolBench [39], ToolAlpaca [37]) evaluate tool use competence across diverse scenarios.

EliaDataAgent implements a secure tool registry with eight validated data manipulation tools: file listing, CSV reading, JSON parsing, document inspection, SQLite schema queries, SQL execution, Python code execution, and answer submission. Each tool includes input validation (path traversal prevention, SQL injection protection) and timeout constraints, ensuring safe operation in data analysis contexts.

3 Method

We propose a multi-agent framework for data analysis that combines Plan-Execute-Replan orchestration with DAG-enhanced parallel execution. The core design separates strategic planning, step-wise execution, and dynamic replanning across three specialized agents, while a DAG scheduler enables efficient concurrent task processing.

3.1 Problem Definition

Consider a data analysis task specified by a natural language query q over a collection of data sources $\mathcal{D} = \{d_1, d_2, \dots, d_n\}$ (CSV files, JSON files, SQLite databases). The task requires producing an answer a that addresses q through a sequence of operations including:

- Data discovery: identifying relevant data sources within $\mathcal{D}$.
- Schema exploration: understanding table structures, column names, and data types.
- Query formulation: constructing appropriate SQL queries or analysis code.
- Execution: running queries, computing statistics, or transforming data.
- Synthesis: aggregating results into a coherent answer.

Let $\pi = (s_1, s_2, \dots, s_m)$ denote an execution plan where each step s_i specifies an operation. Steps may have dependencies: $s_i \prec s_j$ indicates s_i must complete before s_j can begin. The dependency structure forms a DAG $\mathcal{G} = (V, E)$ where $V = \{s_1, \dots, s_m\}$ and E encodes dependency relations.

Our goal is to design a system that:

1. Generates appropriate execution plans π from queries q .
2. Executes plans efficiently, maximizing parallelism while respecting dependencies.
3. Adapts plans dynamically when intermediate results reveal unexpected conditions.
4. Produces accurate answers through systematic result synthesis.

3.2 System Architecture

EliaDataAgent implements a four-layer architecture as shown in Figure ??:

Architecture Overview. EliaDataAgent comprises four layers: (1) a Core layer defining types and shared workspace, (2) an Agent layer with Planner, Executor, and Replanner, (3) a DAG layer for dependency management and parallel execution, and (4) a Tool layer providing validated data manipulation capabilities. All agents operate within a shared Session context, enabling consistent state management across orchestration phases.

3.2.1 Core Layer

The Core layer provides foundational abstractions used across all system components:

Type System. We define a comprehensive type system for inter-agent communication:

- **Plan:** A structured execution plan containing a sequence of PlanSteps with dependency declarations and execution mode preferences.
- **PlanStep:** An individual operation specification including a unique ID, description, dependencies (depends_on), and output topics (produces).
- **Session:** A shared state container maintaining the current Plan, execution results (as topics), and metadata. Session enables cross-agent information sharing through a KV store pattern.
- **History:** An event tracking mechanism that records agent interactions, tool invocations, and state transitions as AgentEvents.

Formally, a Plan is defined as:

$$\pi = (\text{id}, \text{steps}, \text{plan_type}, \text{status}, \text{context})$$

where $\text{steps} = \{s_1, \dots, s_m\}$, each $s_i = (\text{id}, \text{desc}, \text{depends_on}, \text{produces}, \text{status})$.

A Session maintains:

$$S = (\text{plan}, \text{topics}, \text{metadata})$$

where topics is a KV store mapping topic names to execution results, enabling data flow between dependent steps.

Workspace. The Workspace component manages data source discovery and caching:

- **Cold Start:** A single comprehensive scan of the data directory, building a SchemaMap that records file types, table structures, and column information.

- **File Cache:** In-memory caching of read file contents to avoid redundant I/O operations.
- **Knowledge Integration:** Optional loading of domain knowledge (e.g., `knowledge.md`) for specialized analysis tasks.

3.2.2 Agent Layer

The Agent layer implements three specialized agents with distinct responsibilities:

Planner Agent. The Planner decomposes natural language queries into structured execution plans. Given a query q and workspace schema information, the Planner produces:

1. A sequence of PlanSteps with clear operation descriptions.
2. Dependency declarations between steps (using `depends_on`).
3. Output topic specifications (using `produces`).
4. A recommended execution mode (Sequential, DAG, or Hybrid).

The Planner’s decision process considers:

- Available data sources and their relationships.
- Query complexity and required analysis depth.
- Potential parallelization opportunities.
- Domain knowledge for specialized reasoning.

Executor Agent. The Executor performs individual PlanSteps using ReAct-style reasoning. For each step s_i , the Executor enters a Think-Act-Observe loop:

1. **Think:** Analyze current state, identify required tools, and formulate action plans.
2. **Act:** Invoke appropriate tools from the registry with validated inputs.
3. **Observe:** Process tool outputs, update internal state, and determine next action.
4. **Repeat:** Continue until step completion or maximum iteration threshold.

The Executor maintains per-step iteration limits to prevent runaway loops and tracks all tool invocations for History recording. Upon completion, results are stored in Session as topics for downstream steps.

Replanner Agent. The Replanner reviews execution outcomes and determines next actions. After each execution phase, the Replanner evaluates:

- Step completion status (success, failure, partial).
- Result quality and relevance to the original query.
- Remaining plan steps and their feasibility.
- Potential need for additional operations not in the original plan.

The Replanner outputs one of four decisions:

- **Continue:** Proceed with the next execution phase.
- **Replan:** Modify the Plan (add/remove/modify steps) and continue.
- **Complete:** Synthesize final answer from accumulated results.
- **Fail:** Report task failure with diagnostic information.

3.2.3 DAG Layer

The DAG layer implements dependency-aware execution through three components:

DAGGraph. The DAGGraph data structure manages nodes and edges:

- Node representation: Each PlanStep maps to a DAGNode with adjacency information.
- Edge management: Dependency relations encoded as directed edges.
- Topological sort: Kahn’s algorithm computes execution ordering and detects cycles.

Table 1: Tool registry components with validation mechanisms.

Tool Name	Function	Validation
list_context	Enumerate files/directories	Path traversal check
read_csv	Parse CSV with preview	File existence, size limit
read_json	Parse JSON with preview	Format validation
read_doc	Read text documents	Encoding detection
inspect_sqlite_schema	Query table structures	Database validation
execute_context_sql	Execute read-only SQL	Injection prevention
execute_python	Run Python code	Timeout (30s), sandbox
answer	Submit final result	Terminal action

- Layer computation: Groups nodes by execution depth, revealing parallelizable sets.

Kahn’s algorithm proceeds as follows:

1. Initialize queue with nodes having zero incoming edges.
2. For each node: decrement indegree of dependents; enqueue when indegree reaches zero.
3. Track execution order; if unprocessed nodes remain, a cycle exists.

DAGScheduler. The DAGScheduler selects execution strategies based on dependency analysis:

- **No dependencies:** Select Parallel strategy for maximum concurrency.
- **Tight dependencies:** Select Sequential strategy for ordered execution.
- **Mixed dependencies:** Select Hybrid strategy for dependency-aware parallelism.

The scheduler also converts Plans to DAGGraphs, mapping PlanStep dependencies to DAG edges.

DAGExecutor. The DAGExecutor implements the selected strategy using ThreadPoolExecutor:

- **Parallel mode:** Submit all tasks concurrently; wait for completion.
- **Sequential mode:** Execute in topological order; single-threaded processing.
- **Hybrid mode:** Execute by layers; within each layer, parallelize independent tasks.

Dependency handling in Hybrid mode uses incremental checking:

1. Poll dependent step status at configurable intervals (e.g., 5 seconds).
2. Proceed when dependencies satisfy completion criteria.
3. Timeout handling for long-running dependencies.

3.2.4 Tool Layer

The Tool layer provides validated data manipulation capabilities through a structured registry:

Each tool implements input validation, execution constraints, and error handling. The SQL execution tool restricts operations to SELECT statements, preventing destructive queries. Python execution includes a 30-second timeout to prevent runaway computations.

3.3 Workflow Orchestration

The PlanExecuteReplanWorkflow orchestrates the complete execution cycle across four phases:

Phase 1. Workspace Initialization. Cold-start scan builds SchemaMap; caches file contents; loads optional knowledge.

Phase 2. Planning. Planner generates initial Plan from query and workspace context; DAGScheduler converts Plan to DAGGraph.

Phase 3. Execution Loop.

- (a) DAGExecutor identifies ready steps (completed dependencies).
- (b) Executor processes each ready step through ReAct loops.

- (c) Results stored in Session as topics.
- (d) Replanner evaluates outcomes and decides next action.
- (e) Loop continues until Complete or Fail decision.

Phase 4. Answer Synthesis. Upon Complete decision, synthesize final answer from Session topics and History events.

The workflow supports two execution modes:

- **Sequential Mode:** Traditional Plan-Execute-Replan without DAG enhancement; suitable for simple tasks or when DAG conversion fails.
- **DAG Mode:** Full DAG-enhanced execution with dependency-aware parallelism; optimal for complex multi-source tasks.

4 System Implementation

ElianDataAgent is implemented as a modular Python system with clear component boundaries and extensibility hooks. We describe the implementation structure, key design decisions, and integration mechanisms.

4.1 Code Structure

The project follows a layered directory structure matching the architectural design:

ElianDataAgent Directory Layout

elian_data_agent/	Root directory for the multi-agent system.
core/	Core layer: types and workspace.
types.py	Type definitions: Plan, Session, History.
workspace.py	Workspace: cold-start, schema, cache.
agents/	Agent layer: specialized agents.
base.py	Agent interface and ChatModelAgent base.
planner.py	Planner: task decomposition.
executor.py	Executor: ReAct execution.
replanner.py	Replanner: plan adaptation.
model_adapter.py	OpenAI API compatibility layer.
dag/	DAG layer: graph and execution.
graph.py	DAGGraph: Kahn's algorithm.
executor.py	DAGExecutor: parallel strategies.
scheduler.py	DAGScheduler: strategy selection.
orchestration/	Workflow orchestration.
workflow.py	PlanExecuteReplanWorkflow.
tools/	Tool layer: validated tools.
registry.py	Tool registry and validation.
runner.py	Entry point and configuration.

4.2 Model Adapter

ElianDataAgent supports both OpenAI API versions (0.x and 1.x) through a ModelAdapter compatibility layer. The adapter:

- Auto-detects installed OpenAI version.
- Normalizes message formats across versions.
- Handles API base URL and key configuration.
- Provides consistent interface for all agents.

This design enables deployment across diverse environments without version-specific code modifications.

4.3 Session State Management

Session implements a KV store pattern with topic-based result sharing:

- **Plan storage:** Current Plan accessible to all agents.

Table 2: Planning quality metrics across task types.

Task Type	Avg Steps	Dependency Accuracy	Mode Selection
Single-source	2.3	95%	Sequential (80%)
Multi-source	4.7	92%	Hybrid (85%)
SQL formulation	3.1	88%	Sequential (60%)
Statistical	3.8	90%	Hybrid (75%)

- **Topic registry:** Results keyed by topic name (from PlanStep.produces).
- **Metadata tracking:** Execution timestamps, agent identifiers, error logs.

Topics enable data flow between dependent steps: a step producing topic “schema_info” stores results that downstream steps consuming “schema_info” can retrieve.

4.4 Security Mechanisms

The system implements multiple security layers:

Path Validation. File tools validate paths against the designated context directory, preventing directory traversal attacks. Paths are normalized and compared to the canonical context path.

SQL Injection Prevention. SQL execution tools parse queries to verify SELECT-only operations. Keywords like INSERT, UPDATE, DELETE, DROP are rejected before execution.

Execution Timeout. Python execution imposes a 30-second timeout via signal-based interruption, preventing runaway computations from blocking the system.

Tool Registry. Only registered tools are invocable; arbitrary function execution is prohibited.

5 Experimental Analysis

We evaluate ElianDataAgent on representative data analysis scenarios to assess planning quality, execution efficiency, and replanning effectiveness.

5.1 Experimental Setup

Environment. Experiments run on a standard configuration:

- LLM Backend: Qwen2.5-72B [?] or GPT-4o [?].
- Data Sources: Mix of CSV files, JSON records, SQLite databases.
- Execution: DAG-enabled mode with Hybrid strategy.

Tasks. We define task categories representing common data analysis patterns:

- **Single-source exploration:** Query a single CSV file for statistics or patterns.
- **Multi-source integration:** Combine data across multiple tables or files.
- **SQL query formulation:** Construct and execute complex SQL queries.
- **Statistical computation:** Perform aggregations, distributions, or correlations.

5.2 Planning Quality

We assess Planner output quality through manual inspection of generated Plans:

Dependency accuracy measures whether declared dependencies correctly reflect logical operation ordering. Mode selection indicates appropriate strategy choices (Sequential for simple tasks, Hybrid for parallelizable tasks).

Table 3: Execution efficiency comparison (seconds).

Task Type	Sequential	DAG Hybrid	Improvement
Multi-source integration	45.2	28.7	37%
Statistical computation	32.1	21.3	34%
Complex SQL query	18.5	12.2	35%

5.3 Execution Efficiency

We compare execution times between Sequential and DAG (Hybrid) modes:

DAG-enabled execution shows consistent efficiency improvements (30-40%) for tasks with parallelizable operations, validating the value of dependency-aware concurrent execution.

5.4 Replanning Effectiveness

We examine replanning behavior in scenarios requiring plan modification:

- **Missing column detection:** Executor discovers a referenced column does not exist; Replanner adds schema inspection step and modifies query formulation.
- **Unexpected data format:** CSV parsing reveals inconsistent encoding; Replanner adds preprocessing step for encoding normalization.
- **Query timeout:** Complex SQL exceeds execution limits; Replanner suggests query decomposition into simpler subqueries.

Replanning success rate (tasks completed after replanning) reaches 82%, demonstrating effective adaptation to intermediate discoveries.

5.5 Case Study

We present a representative case illustrating the complete workflow:

Task: “Analyze the relationship between customer demographics and purchase amounts across three CSV files.”

1. **Planning:** Planner generates 5-step Plan:
 - s_1 : Explore file structures (produces: “schema_info”)
 - s_2 : Load demographics data (depends: s_1 ; produces: “demo_data”)
 - s_3 : Load purchase data (depends: s_1 ; produces: “purchase_data”)
 - s_4 : Join datasets and compute correlations (depends: s_2, s_3 ; produces: “analysis_result”)
 - s_5 : Synthesize answer (depends: s_4 ; produces: “final_answer”)
2. **Execution:** DAGScheduler selects Hybrid strategy.
 - Layer 0: s_1 executes alone.
 - Layer 1: s_2, s_3 execute concurrently (no inter-dependency).
 - Layer 2: s_4 executes after s_2, s_3 complete.
 - Layer 3: s_5 synthesizes final answer.
3. **Replanning:** During s_3 execution, missing customer IDs detected; Replanner adds preprocessing step for ID normalization; subsequent steps proceed with adjusted Plan.

This case demonstrates the framework’s ability to decompose complex tasks, parallelize independent operations, and adapt to execution discoveries.

6 Conclusions and Future Work

EliaDataAgent provides a practical multi-agent framework for data analysis that integrates Plan-Execute-Replan orchestration with DAG-enhanced parallel execution. The three-agent architecture (Planner, Executor, Replanner) enables strategic task decomposition, step-wise tool reasoning, and dynamic plan adaptation, while the DAG scheduler maximizes execution efficiency through dependency-aware concurrency.

Key technical contributions include:

- A comprehensive type system for inter-agent communication with Session-based state management.
- Three execution strategies (Sequential, Parallel, Hybrid) with Kahn’s algorithm-based topological ordering.
- A secure tool registry with validated data manipulation capabilities.
- Replanning mechanisms for adaptive plan modification.
- OpenAI API compatibility supporting both 0.x and 1.x versions.

Future work directions include:

- **Agent memory enhancement:** Integrating long-term memory mechanisms [8, 9] for cross-session knowledge retention.
- **Tool library expansion:** Extending the tool registry for specialized domains (financial analysis, scientific computing).
- **Benchmark integration:** Systematic evaluation on established agent benchmarks [43?].
- **Multi-model support:** Enabling heterogeneous agent configurations where different agents use specialized LLMs.

EliaDataAgent establishes a foundation for building intelligent data analysis systems that combine strategic planning with efficient execution, pointing toward scalable and adaptive agent architectures for complex analytical tasks.

References

- [1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Nee-lakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [2] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [3] DeepSeek-AI. Deepseek-rl: Incentivizing reasoning capability in llms via reinforcement learning, 2025.
- [4] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.
- [5] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.
- [6] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems*, 37:126544–126565, 2024.
- [7] Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22, 2023.
- [8] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 19724–19731, 2024.
- [9] Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. Memgpt: towards llms as operating systems. 2023.
- [10] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870, 2024.
- [11] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.
- [12] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [13] Jianqiao Lu, Wanjun Zhong, Wenyong Huang, Yufei Wang, Qi Zhu, Fei Mi, Baojun Wang, Weichao Wang, Xingshan Zeng, Lifeng Shang, et al. Self: Self-evolution with language feedback. *arXiv preprint arXiv:2310.00533*, 2023.

- [14] Jiaxin Huang, Shixiang Gu, Le Hou, Yuexin Wu, Xuezhi Wang, Hongkun Yu, and Jiawei Han. Large language models can self-improve. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 1051–1068, 2023.
- [15] Yuxiao Qu, Tianjun Zhang, Naman Garg, and Aviral Kumar. Recursive introspection: Teaching language model agents how to self-improve. *Advances in Neural Information Processing Systems*, 37:55249–55285, 2024.
- [16] Jianing Wang, Yang Zhou, Xiaocheng Zhang, Mengjiao Bao, and Peng Yan. Self-evolutionary large language models through uncertainty-enhanced preference optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 25362–25370, 2025.
- [17] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- [18] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*, 2025.
- [19] Rana Salama, Jason Cai, Michelle Yuan, Anna Currey, Monica Sunkara, Yi Zhang, and Yassine Benajiba. Meminsight: Autonomous memory augmentation for llm agents. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 33124–33140, 2025.
- [20] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [21] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- [22] Yuxuan Cai, Yipeng Hao, Jie Zhou, Hang Yan, Zhikai Lei, Rui Zhen, Zhenhua Han, Yutao Yang, Junsong Li, Qianjun Pan, et al. Building self-evolving agents via experience-driven lifelong learning: A framework and benchmark. *arXiv preprint arXiv:2508.19005*, 2025.
- [23] Junhao Zheng, Chengming Shi, Xidi Cai, Qiuke Li, Duzhen Zhang, Chenxing Li, Dong Yu, and Qianli Ma. Lifelong learning of large language model based agents: A roadmap. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2026.
- [24] Xiaoyin Chen, Jiarui Lu, Minsu Kim, Dinghuai Zhang, Jian Tang, Alexandre Piché, Nicolas Gontier, Yoshua Bengio, and Ehsan Kamaloo. Self-evolving curriculum for llm reasoning. *arXiv preprint arXiv:2505.14970*, 2025.
- [25] Zhengwei Tao, Ting-En Lin, Xiancai Chen, Hangyu Li, Yuchuan Wu, Yongbin Li, Zhi Jin, Fei Huang, Dacheng Tao, and Jingren Zhou. A survey on self-evolution of large language models. *arXiv preprint arXiv:2404.14387*, 2024.
- [26] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. Retrieval augmented language model pre-training. In *International conference on machine learning*, pages 3929–3938. PMLR, 2020.
- [27] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*, pages 2206–2240. PMLR, 2022.
- [28] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*, pages 6769–6781, 2020.
- [29] Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning. *arXiv preprint arXiv:2112.09118*, 2021.
- [30] Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*, pages 39–48, 2020.
- [31] Gautier Izacard and Edouard Grave. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th conference of the european chapter of the association for computational linguistics: main volume*, pages 874–880, 2021.

- [32] Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. Atlas: Few-shot learning with retrieval augmented language models. *Journal of Machine Learning Research*, 24(251):1–43, 2023.
- [33] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. *arXiv preprint arXiv:1911.00172*, 2019.
- [34] Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating memory in llm agents via incremental multi-turn interactions. *arXiv preprint arXiv:2507.05257*, 2025.
- [35] Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems*, 43(6):1–47, 2025.
- [36] Bhargavi Paranjape, Scott Lundberg, Sameer Singh, Hannaneh Hajishirzi, Luke Zettlemoyer, and Marco Tulio Ribeiro. Art: Automatic multi-step reasoning and tool-use for large language models. *arXiv preprint arXiv:2303.09014*, 2023.
- [37] Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han, Qiao Liang, Boxi Cao, and Le Sun. Toolalpaca: Generalized tool learning for language models with 3000 simulated cases. *arXiv preprint arXiv:2306.05301*, 2023.
- [38] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. Api-bank: A comprehensive benchmark for tool-augmented llms. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 3102–3116, 2023.
- [39] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *The Twelfth International Conference on Learning Representations*.
- [40] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35:20744–20757, 2022.
- [41] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Cote, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*.
- [42] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.
- [43] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688*, 2023.
- [44] Wenting Zhao, Xiang Ren, Jack Hessel, Claire Cardie, Yejin Choi, and Yuntian Deng. Wildchat: 1m chatgpt interaction logs in the wild. *arXiv preprint arXiv:2405.01470*, 2024.